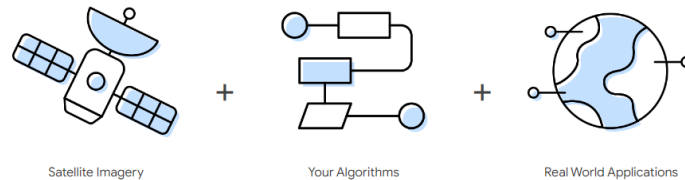


Pengenalan Google Earth Engine Untuk Perencanaan

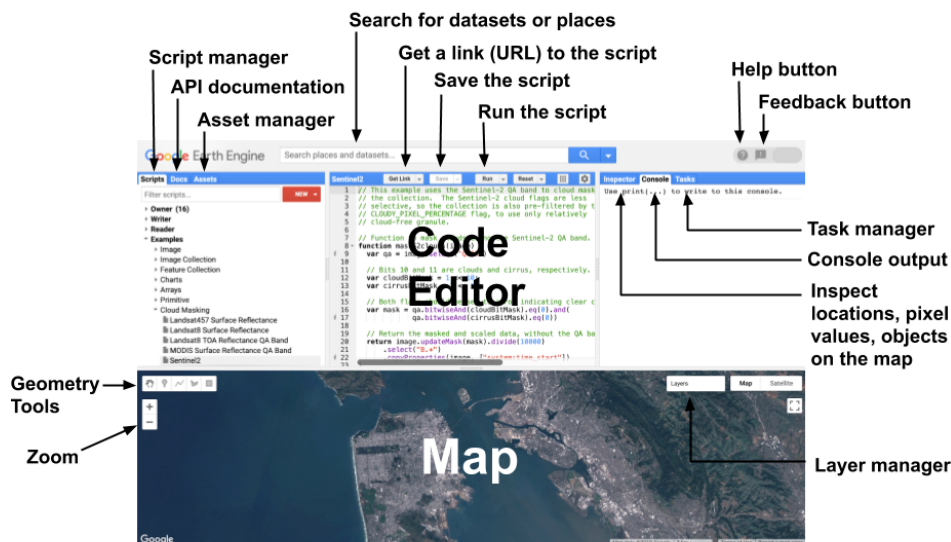
Meet Earth Engine

Google Earth Engine combines a multi-petabyte catalog of satellite imagery and geospatial datasets with planetary-scale analysis capabilities. Scientists, researchers, and developers use Earth Engine to detect changes, map trends, and quantify differences on the Earth's surface. Earth Engine is now available for commercial use, and remains free for academic and research use.



Perencanaan memerlukan berbagai macam jenis data agar menghasilkan rencana yang komprehensif, integratif dan efektif terutama dalam perencanaan tata ruang. Dalam rangka melakukan analisis keruangan, data spasial merupakan salah satu data yang paling dibutuhkan. Data spasial adalah data yang memiliki atribut spasial yang menunjukkan lokasi, letak dan posisinya di permukaan bumi (Yeung & Hall, 2014). Ketersediaan data spasial saat ini sudah sangat melimpah, banyak diantaranya bersifat *open source* sehingga dapat diakses oleh siapapun secara mudah dan cepat.

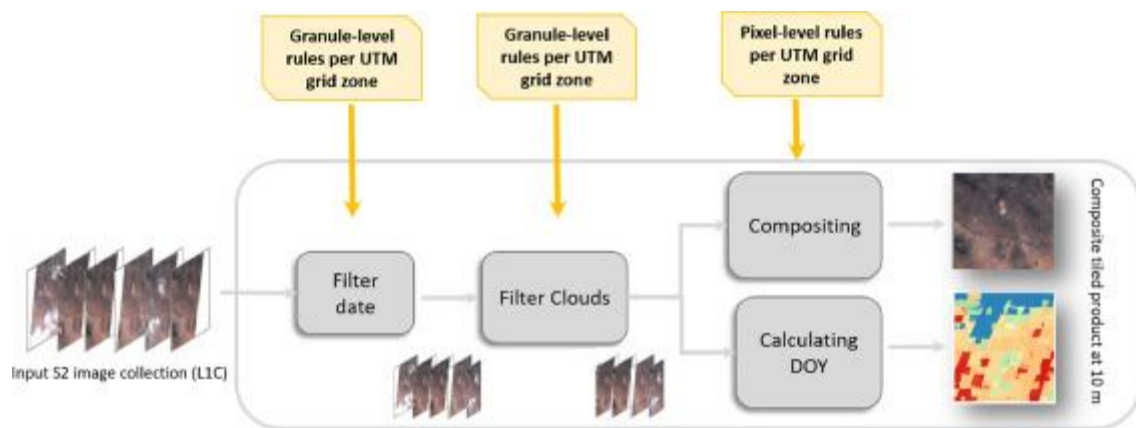
Perkembangan teknologi Geographic Information System (GIS) cukup pesat dan memberikan berbagai kemudahan bagi penggunaannya. Google Earth Engine (GEE) merupakan salah satu platform yang memberikan kemudahan terhadap analisis dan pengolahan data spasial melalui penyediaan data yang besar (*big data*) secara kontinu melalui sistem *cloud* (S. M. Gorelick, 2015). GEE merupakan platform yang menyediakan data geospasial dan citra penginderaan jauh sekaligus menggabungkan fungsi pengolahan data spasial melalui berbagai algoritma machine learning serta dapat melakukan visualisasi data.



Gambar Komponen dari Earth Engine Code Editor

Earth Engine (EE) Code Editor merupakan interface GEE yang memiliki berbagai *tools* untuk menjalankan berbagai perintah. Fitur Code Editor dirancang untuk mengakomodir pekerjaan secara mudah dan cepat dengan data geospasial yang kompleks. Pengguna dapat menuliskan perintah dalam bentuk JavaScript pada Code Editor untuk memanggil, mengolah dan menampilkan data. Perlu diingat bahwa platform GEE merupakan web-based dan cloud-based platform sehingga hanya dapat bekerja apabila terkoneksi dengan jaringan internet.

Proses



Tutorial 1 Akuisisi Citra Sentinel 2A True Color Composite 432

Langkah 1 Login GEE

Login menggunakan akun yang sudah terdaftar Google Cloud project. Berikut informasi mengenai [Earth Engine Cloud project setup guide](#).

<https://code.earthengine.google.com>

Langkah 2 Membuka katalog data

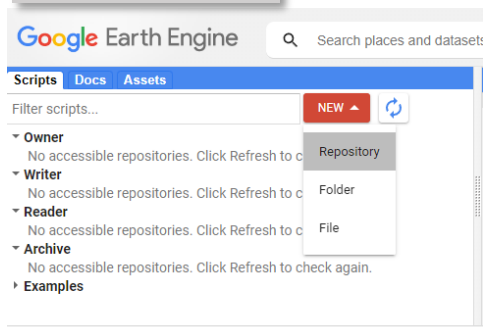
Earth Engine Data Catalog



Sebelum memanggil data dan melakukan analisis terlebih dahulu membuka direktori data untuk mengetahui ketersediaan data dan detail keterangan dari data yang akan dipanggil.

<https://developers.google.com/earth-engine/datasets>

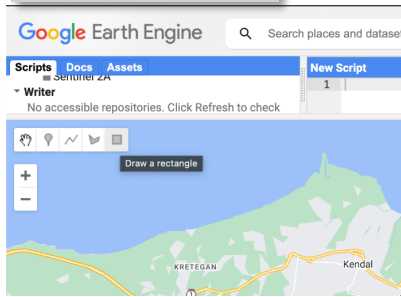
Langkah 3 Membuat new repository



Buat *Repository* baru untuk menyimpan script yang digunakan. **Script > New > Repository**

Langkah 4

Membuat AOI (*Area Of Interest*)



Buat Polygon sebagai AOI. **Draw a rectangle** > Arahkan cursor pada area kajian > **Rename**

Langkah 5

Memanggil data

Data yang akan digunakan adalah citra Sentinel 2A

```
var dataset = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
```

Langkah 5

Filter data

Masukkan rentang waktu perekaman citra yang diinginkan, persentase awan dan area kajian

```
var dataset = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterDate('2024-01-01', '2024-07-31') //rentang perekaman
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE',20)) //persentase awan
    .filterBounds(aoi) //memilih data yg masuk AOI
```

Langkah 6

Cek ketersediaan data

Memastikan data tersedia sehingga dapat dilakukan proses lanjutan

```
print (dataset)
```

Langkah 7

Memilih data yang akan digunakan

Opsi yang dapat digunakan; *first()*, *mean()*, *median()* dsb

```
var dataset = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterDate('2024-01-01', '2024-07-31') //rentang perekaman
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE',20)) //persentase awan
    .filterBounds(aoi) //memilih data yg masuk AOI
    .median() //memilih nilai tengah dr data yg tersedia
    .clip(aoi);
```

Langkah 8

Visualisasi data

Diperlukan parameter visualisasi untuk dapat menampilkan data

```
var imageviz = {min:1, max:3, bands:['B4','B3','B2']};
```

Langkah 9

Tampilkan data

Menampilkan data pada halaman utama sesuai parameter visualisasi, kemudian atur tampilan agar fokus pada AOI

```
Map.addLayer (dataset, imageviz, 'Citra Sentinel 2024')  
Map.centerObject (aoi, 10)
```

Langkah 10

Sesuaikan ulang parameter visualisasi

Menyesuaikan kembali parameter visualisasi dengan mengubah rentang nilai min & max melalui **Setting > Stretch 100% > Ubah nilai imageviz sesuai nilai yg muncul**

Langkah 11

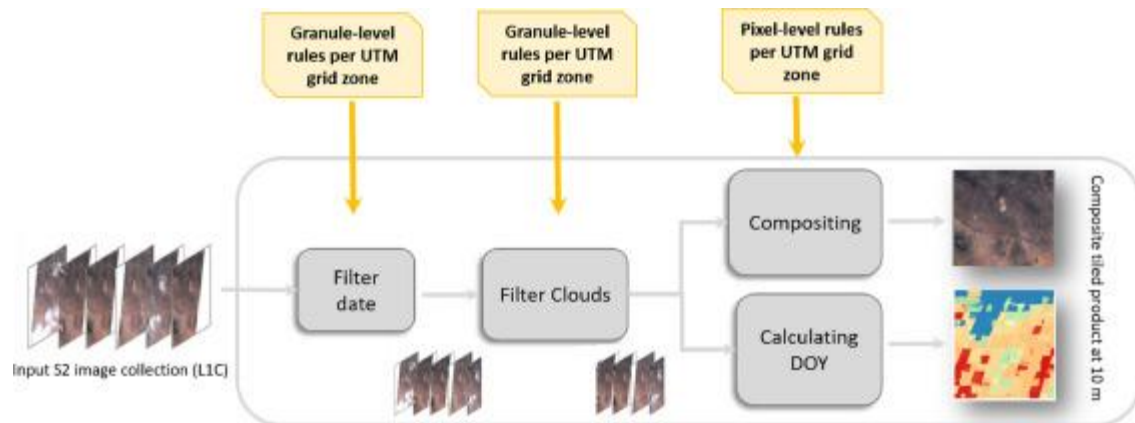
Exporting data

Sebelum export data terlebih dahulu memilih band yang dibutuhkan, Optical bands dari Citra Sentinel 2A adalah B4 (red), B3 (green), B2 (blue)

```
var selectband = ['B4','B3','B2'];
```

```
Export.image.toDrive({  
  image: dataset.clip(aoi).select(selectband),  
  folder: 'Export GEE',  
  description: 'Citra Sentinel 2A',  
  scale: 10,  
  region: aoi});
```

Tutorial 2 Extracting LULC Map



Langkah 1

Cloud & Cirrus Masking

Membuat *function mask* untuk menghilangkan awan pada dataset yang digunakan. *Function* ini khusus untuk Sentinel-2, apabila menggunakan citra lain maka terapkan *function* yg sesuai.

```
/**
 * Function to mask clouds using the Sentinel-2 QA band
 * @param {ee.Image} image Sentinel-2 image
 * @return {ee.Image} cloud masked Sentinel-2 image
 */
function maskS2clouds(image) {
  var qa = image.select('QA60');

  // Bits 10 and 11 are clouds and cirrus, respectively.
  var cloudBitMask = 1 << 10;
  var cirrusBitMask = 1 << 11;

  // Both flags should be set to zero, indicating clear conditions.
  var mask = qa.bitwiseAnd(cloudBitMask).eq(0)
    .and(qa.bitwiseAnd(cirrusBitMask).eq(0));

  return image.updateMask(mask).divide(10000);
}
```

Langkah 2

Menerapkan masking pada dataset

Memasukkan *function* dalam script dataset dan menampilkan hasil penerapan *function*, cek apakah masih terdapat awan, apabila masih dijumpai awan maka bisa dicoba merubah persentase awan

```
var dataset = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterDate('2024-01-01', '2024-07-31') //rentang perekaman
  // Pre-filter to get less cloudy granules.
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE',20)) //persentase awan
  .filterBounds(aoi) //memilih data yg masuk AOI
  .map(maskS2clouds) //menerapkan masking
  .median(); //memilih nilai median dr dataset

var visualization = {
  min: 0.0,
  max: 0.3,
  bands: ['B4', 'B3', 'B2'],
};
```

```
Map.addLayer(dataset.clip(aoi), visualization, 'Citra Sentinel 2A Masking');
```

Langkah 3

Area dan Sample Processing

Perlu membuat area processing dan sample processing sebagai salah satu input dalam algoritma klasifikasi. Pada tahap dimasukkan variable berupa band apa saja yang akan dijadikan dasar untuk generating LULC Classification.

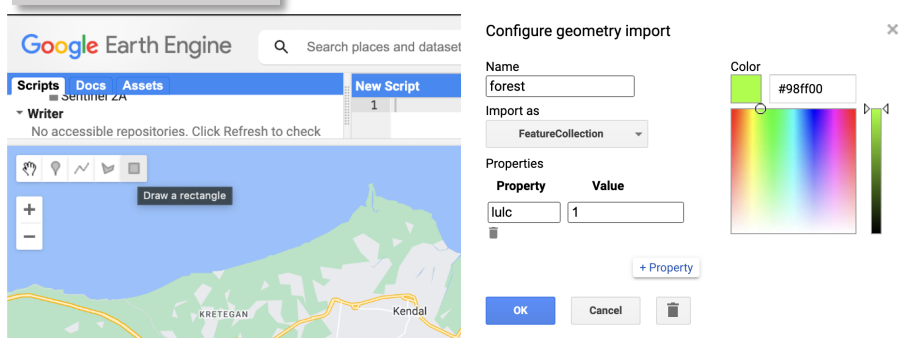
```
//Area Processing//
var s2area = dataset.clip(aoi);

// Sample Processing//
//Select bands for training
var s2bands = ['B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8', 'B11', 'B12']
var s2select = s2area.select(s2bands);

print ('s2 selectbands=', s2select); //menampilkan band yang terpilih
```

Langkah 4

Membuat sample (data training)



Klik “Draw a rectangle” > Klik “Geometry Import” > Klik “Setting” > isikan kelas LULC pada “Name”, import as pilih “FeatureCollection”, isikan Property “lulc” dan value. Kemudian buat script variable sample.

```
//Merge all training data
var kelas = water.merge(forest).merge(builtup).merge(paddy).merge(dryland); //5 kelas lulc

print ('Kelas=',kelas); //menampilkan kelas yang dibuat
```

Langkah 5

Membuat sample (data training)

Sample akan digunakan untuk menjalankan algoritma klasifikasi, dibutuhkan training sample dan testing sample sehingga total sample dibagi (*split*) menjadi 2 dengan proporsi yang ditentukan.

```
var samples = s2select.sampleRegions({
  collection: kelas,
  properties: ['lulc'],
  scale: 20
}).randomColumn('random');

// Split sample
// 70:30
var split = 0.7; // proporsi split sample
var training = samples.filter(ee.Filter.lt('random', split)); // less than 0,7 (70% dr total sample),
var testing = samples.filter(ee.Filter.gte('random', split)); // greater than 0,7 (30% dr total sample),

print (
  ee.String('Total sample=').cat(ee.String(samples.aggregate_count('.all'))),
  ee.String('Training n=').cat(ee.String(training.aggregate_count('.all'))),
  ee.String('Testing n=').cat(ee.String(testing.aggregate_count('.all'))),
  );
```

Langkah 6

Memilih algoritma dan menjalankan klasifikasi

Algoritma klasifikasi dapat dilihat pada “Docs” > “ee.Classifier”. Algoritma yang akan digunakan dalam tutorial ini adalah Random Forest Classifier (*ee.Classifier.smileRandomForest*).

```
// Random Forest Classifier
var classifier = ee.Classifier.smileRandomForest(100).train({
  features: training,
  classProperty: 'lulc',
  inputProperties: s2bands});

//Classify the input imagery
var classmap = s2select.classify(classifier);
```

Langkah 7

Menyusun color palette

Color palette akan digunakan untuk menampilkan hasil klasifikasi, masing-masing kelas diberi warna disesuaikan dengan kebutuhan user. Dicontohkan pada tahap ini terdapat 5 kelas LULC sehingga *color palette* didefinisikan menjadi 5 warna.

```
// Define a palette for the Land Use classification.
var palette = [
  'blue', // water (0),
  'green', // forest (1),
  'red', // builtup (2),
  'yellow', // paddy (3),
  'orange', // dryland (4)
];

Map.addLayer(classmap, {min:0, max:4, palette: palette}, 'Land Use 2024');
```

Langkah 8

Menghitung akurasi klasifikasi

Menyusun variable untuk mengetahui seberapa nilai akurasi klasifikasi yg dijalankan. Dicontohkan dua ukuran akurasi yaitu Overall Accuracy dan Kappa Coefficient

```
// Accuracy Assesment RF
var confusionMatrix = ee.ConfusionMatrix(testing.classify(classifier)
  .errorMatrix({
    actual: 'lulc',
    predicted: 'classification'
  }));

print (
  ee.String('Overall Accuracy= ').cat(ee.String(confusionMatrix.accuracy())),
  ee.String('Kappa Coefficient= ').cat(ee.String(confusionMatrix.kappa()))
);
```

Langkah 9

Membuat variable legenda

Untuk dapat menampilkan hasil klasifikasi maka harus membuat variable visualisasi

```
//-----Create Legend-----//
var legend = ui.Panel({
  style: {
    position: 'bottom-right',
    padding: '8px 15px'
  }
});

var colors = ['blue','green','red','yellow','orange'];
var names = ['Tubuh Air', 'Hutan', 'Lahan Terbangun', 'Sawah', 'Lahan Kering'];

//Create and add the legend title
var legend = ui.Panel({style:{position: 'bottom-right'}});
legend.add(ui.Label({
  value: "Land Cover",
  style: {
    fontWeight: 'bold',
    fontSize: '18px',
    margin: '0 0 4px 0',
    padding: '0px'
  }
}));
```

```

var entry;
for (var x = 0; x<=4; x++){
  entry = [
    ui.Label({style:{color:colors[x],margin: '0 0 4px 0'}, value:'█'}),
    ui.Label({
      value: names[x],
      style: {
        margin: '0 0 4px 4px'
      }
    })
  ];
  legend.add(ui.Panel(entry, ui.Panel.Layout.Flow('horizontal')));
}

```

Map.add(legend);

Langkah 11

Exporting data

```

Export.image.toDrive({
  image: classmap,
  folder: 'Export GEE',
  description: 'LULC Map 2024',
  scale: 20,
  region: aoi});

```